
Jasper OS v2.0: Base44 Ω -Manifold Deployment Map

Implementation Reference — Version 2.0.0 RC

Date: July 2026

Jasper OS Research Collective

Abstract

This document provides a complete implementation reference mapping each visible surface of the Base44 platform runtime to its corresponding Ω -Manifold operator defined in the Jasper OS v2.0 white paper. The Base44 desktop environment — deployed at app.base44.com — serves as the physical substrate on which the eight-component manifold $\Omega = (M\partial \times MH^n \times MF \times M\Phi \times M\Psi \times MH_k \times MJ \times M\hat{H}^c)/\sim$ is instantiated. We document operator-to-surface bindings for all eight components: the geometric cognition substrate (∂), multi-agent operating system (H^n), topological knowledge engine (F), gauge-theoretic memory system (Φ), Hamiltonian coordination engine (Ψ), teleportable identity layer (H_k), self-improving capability manifold (J), and auditable AGI shell ($\hat{H}^c$). The HyperChain registry protocol anchors identity barcodes and Lean 4 proof certificates on-chain. A ten-step bootstrap protocol

and five formal guarantees are provided. A live demonstration is publicly accessible via the hyperlink in Section 4.

Keywords: Ω -Manifold; Jasper OS; Base44; topological knowledge engine; Riemannian manifold; HyperChain registry; gauge-theoretic memory; multi-agent systems; Lean 4.

Table of Contents

- Section 1 — Executive Summary
 - Section 2 — Base44 Runtime Surface Map
 - Section 3 — Operator Instantiation Table
 - Section 4 — Layer Stack and Live Demo Access
 - Section 5 — Inter-Component Interface Specifications
 - Section 6 — HyperChain Registry Protocol
 - Section 7 — Bootstrap Protocol
 - Section 8 — Formal Guarantees
 - Appendix B — Operator Quick Reference
-

Section 1 — Executive Summary

Jasper OS v2.0 implements a mathematically rigorous cognitive substrate grounded in differential geometry, algebraic topology, gauge theory, and Hamiltonian mechanics. The Ω -Manifold is the top-level abstraction — a product manifold of eight specialized sub-systems, each corresponding to a distinct mathematical discipline and a distinct runtime surface within the Base44 platform.

Base44 (app.base44.com) provides the deployment runtime: a full-stack application platform with serverless functions, managed databases, MCP server integration, OAuth authentication, GitHub synchronization, and an AI-driven prompt interface. Each of these surfaces is formally identified with one of the eight Ω -Manifold operators in the sections below.

Summary Statistics

- **8 Operators** — Fully Mapped
- **20 Milestones** — Implementation Checklist
- **5 Theorems** — Formal Guarantees
- **1 Functor** — $F: \text{Seeds} \rightarrow \text{Systems}$

Section 2 — Base44 Runtime Surface Map

The following table maps each physically observable surface of the Base44 platform to its corresponding Ω -Manifold operator, mathematical domain, and governing equation.

Operator	Name & Base44 Surface	Mathematical Domain & Key Equation	§
∂	Geometric Cognition Substrate — <i>src/pages/</i> and <i>src/entities/</i>	Riemannian manifold (M,g) ; $R(u,v)w = \nabla_u \nabla_v w - \nabla_v \nabla_u w - \nabla[u,v]w$	§3
H^n	Multi-Agent Operating System — <i>src/api/</i> and MCP	Cohomology $H^n(M,\mathbb{R}) = \ker(\delta)/\text{im}(\delta)$; coalition: $\varphi \sim \psi \in$	§4

Operator	Name & Base44 Surface	Mathematical Domain & Key Equation	§
	Server	H^{n+m}	
F	Topological Knowledge Engine — Prompt Bar and <i>src/api/</i>	Functor F: Seeds→Systems; $F(G_s) = \int P(G G_s)dG$; Vietoris-Rips filtration	§5
Φ	Gauge-Theoretic Memory System — <i>src/lib/</i> , <i>src/components/</i> , Managed DB	Principal bundle $P(M,G)$; holonomy $\text{Hol}_\gamma(A) = \mathcal{P}\text{-exp}(-\oint_\gamma A)$	§6
Ψ	Hamiltonian Coordination Engine — <i>src/functions/</i> Serverless	Cotangent bundle T^*M ; $dq/dt = \partial H/\partial p$, $dp/dt = -\partial H/\partial q$; leapfrog integration	§7
H_k	Teleportable Identity Layer — OAuth: Google / Apple / GitHub	Homology $H_k(M,\mathbb{Z})$; quasi-isomorphism $f_*([M_A]) = [M_B]$	§8
J	Self-Improving Capability Manifold — GitHub Sync + Activity Monitor	Capability manifold C ; Jacobian $J=Df$; $dc/dt = J^T \cdot \nabla P(c)$	§9
Ĥ^c	Auditable AGI Shell — Code Tab, Docs MCP, Deploy Pipeline	Audit algebra $\mathcal{A}=(O,\leq,\vdash)$; $\hat{H}^c = H + V_{\text{audit}}$; $V_{\text{audit}}=+\infty$ on non-certified states	§10

Table 1. Base44 runtime surface to Ω -Manifold operator bindings.

Section 3 – Operator Instantiation Table

Table 2 provides the complete instantiation specification for each operator including hardware targets and white paper section references.

Symbol	Name	Hardware Target	Key Equation	Section
∂	Geometric Cognition Substrate	GPU Tensor Cores	$R(u,v)w=\nabla_u\nabla_v w-\nabla_v\nabla_uw$	§3
H^n	Multi-Agent OS	Multi-core CPU	$H^n(M,\mathbb{R})=\ker(\delta)/\text{im}(\delta)$	§4
F	Topological Knowledge Engine	GPU+CPU Hybrid	$F(G_s)=\int P(G G_s)dG$	§5
Φ	Gauge-Theoretic Memory	High-Bandwidth Memory (HBM)	$\text{Hol}_\gamma(A)=\mathcal{P}e^{xp(-\oint_\gamma A)}$	§6
Ψ	Hamiltonian Coordination	TPU / Symplectic Hardware	$\begin{aligned}dq/dt &= \partial H/\partial p; \\ dp/dt &= -\partial H/\partial q\end{aligned}$	§7
H_k	Teleportable Identity Layer	NVMe SSD (barcode archive)	$f^*([M_A])=[M_B]$	§8
J	Self-Improving Capability	NAS Cluster (distributed GPU)	$\begin{aligned}dc/dt &= J^T \cdot \nabla P(c)\end{aligned}$	§9
$\hat{H}^c$	Auditable AGI Shell	Trusted Execution Environment	$\hat{H}^c = H + V_{\text{audit}}$	§10

Table 2. Complete operator instantiation with hardware targets.

Section 4 — Layer Stack and Live Demo

Access

The Ω -Manifold is organized as a hierarchical stack of eight layers. The base layer (∂) provides the geometric substrate; each successive layer builds on those below. The outermost layer ($\hat{H}^c$) acts as a universal audit gate ensuring all outputs are formally certified before external exposure.

Layer Stack (top to bottom)

[$\hat{H}^c$]

AUDIT AND VERIFICATION — Mathematically Auditable AGI Shell

[J] META-LEARNING / NAS — Self-Improving Capability Manifold

[H_k] HOMOLOGICAL IDENTITY — Teleportable Identity Layer

[Ψ] ENERGY-STATE SCHEDULING — Hamiltonian Coordination Engine

[Φ] FIBER BUNDLE MEMORY — Gauge-Theoretic Memory System

[F] PERSISTENT HOMOLOGY KB — Topological Knowledge Engine

[H^n] AGENT SCHEDULING / IPC — Multi-Agent Operating System

[∂]

BASE RIEMANNIAN MANIFOLD — Geometric Cognition Substrate (foundation)

Figure 1. Ω -Manifold layer stack — outermost audit layer at top, geometric base at bottom.

Live Demonstration Access

The implementation described in this paper is actively deployed and publicly accessible. Readers are encouraged to explore the live system to verify operator bindings, inspect the HyperChain registry, and interact with the Base44 runtime environment directly.

Access the Live Deployment:

Access Live Demo: app.base44.com

Note: Authentication is required via Google, Apple, or GitHub OAuth. All HyperChain registry entries generated during the session are publicly verifiable.

Section 5 — Inter-Component Interface Specifications

Each pair of adjacent Ω -Manifold layers communicates through a formally specified interface. Table 3 enumerates all inter-component interfaces, their mathematical channels, and the constraints that must hold for system consistency.

Source → Target	Interface Type	Mathematical Channel	Constraint
$\partial \rightarrow \mathbf{H}^n$	Cochain injection	$v \in T_{pM}$ embedded as 1-	$\delta\phi_v=0$ iff v is Killing

Source $\rightarrow$ Target	Interface Type	Mathematical Channel	Constraint
		cochains via $v \mapsto g(v, -)$	
$\partial \rightarrow \Phi$	Fiber bundle projection	$\pi: P(M, G) \rightarrow M$; M is base space of memory bundle	Transition functions smooth
$H^n \rightarrow \Psi$	Phase-space lift	Harmonic cochains mapped to $(q, p) \in T^*M$	$\{H_A, H_B\} = 0$ non-interacting
$\Phi \rightarrow F$	Holonomy to graph weight	$w = \text{Tr}(\text{Hol}(A)) / \dim(G) \in [0, 1]$	Triangle inequality holds
$\Psi \rightarrow J$	Flow to gradient field	X_H projected onto ∇P via Poisson map	Must be Poisson map
$H_k \rightarrow \hat{H}^c$	Certificate injection	Barcodes serialized as Lean 4 structures	Valid Lean 4 isomorphism proof
$J \rightarrow \partial$	Metric tensor update	$\Delta c \mapsto \Delta g = \rho(\Delta c) \in \text{Sym}^2(T^*M)$	g remains positive-definite
$\hat{H}^c \rightarrow \mathbf{All}$	Universal audit gate	$\forall o: (o, \pi_o) \in \text{Output} \times \text{Proof}$ before exposure	$\forall o, \exists \pi_o: \not\models \pi_o: \text{spec}(o)$

Table 3. Inter-component interface specifications (§12 of white paper).

Section 6 — HyperChain Registry Protocol

The HyperChain registry provides tamper-evident, on-chain anchoring for four categories of Jasper OS runtime events. All chain commitments use Keccak-256. Proof certificates are generated and verified via the Lean 4 kernel.

Entry Type	Operator	Trigger	Payload (key fields)	Chain Commitment
Identity Barcode Anchoring	H_k	OAuth login / cross-deployment migration	barcode_hash: SHA-256(H_k); bottleneck_dist: ε ; provider: google apple github	Keccak-256(barcode timestamp deployment_id)
Proof Certificate Registry	$\hat{H}^c$	Every component output before external exposure	proof_term: π_o ; spec_hash: SHA-256(spec(o)); V_audit: 0	Keccak-256(proof_hash op_id manifold_state)
Reconstruction Event Log	F	Prompt bar submission / seed ingestion	seed_graph: G_s ; dgm_hash: SHA-256(Dgm(G_s)); system_id: Σ	Keccak-256(seed_hash functor_trace system_hash)
Capability Snapshot Ledger	J	Each GitHub sync / capability flow step	capability_entropy: H_c ; jacobian_norm: $\ J\ $; improvement_delta: ΔC	Keccak-256(cap_state step_idx perf_score)

Table 4. HyperChain registry entry types and on-chain commitment schema.

Registry Schema

```
HyperChain Registry Entry {
  entry_id:      UUID v4
  operator:      Enum[ $\partial$ ,  $H^n$ ,
    F,  $\Phi$ ,  $\Psi$ ,  $H_k$ , J,  $\hat{H}^c$ ]
  base44_surface: String
  deployment_id: String
  payload_hash:   Keccak-256
  chain_commitment: Keccak-256
  timestamp:
```

```
ISO 8601 (UTC) lean4_proof_ref: String | null status: Enum[PENDING,
ANCHORED, VERIFIED, QUARANTINED] }
```

Listing 1. HyperChain registry entry schema.

Section 7 — Bootstrap Protocol

The following ten-step protocol describes the complete initialization sequence from a minimal seed graph G_s to a fully instantiated Ω -Manifold deployment on Base44. Steps execute sequentially; each step's output is the input to the next.

Step	Operator	Title	Operation	Base44 Surface
1	F	Seed Ingestion	Parse G_s via Vietoris-Rips filtration	Prompt bar
2	∂	Manifold Initialization	Fit metric g to Dgm embedding; initialize Levi-Civita connection	src/entities/schema
3	H^n	Agent Spawning	Compute $H^1(M)$ via Laplacian eigendecomposition; spawn harmonic agents	src/api/ + MCP tools
4	F	Knowledge Expansion	Apply $F_{\text{topo}}: G_s \rightarrow G_{\text{full}}$ via	AI prompt $\rightarrow$ full data model

Step	Operator	Title	Operation	Base44 Surface
			bottleneck minimization	
5	Φ	Memory Materialization	Initialize gauge field A; minimize L_{YM} ; apply Coulomb gauge fixing	Managed DB + src/lib/
6	Ψ	Hamiltonian Calibration	Fit $V(q)$ to task landscape; begin leapfrog integration	src/functions/serverless
7	H_k	Identity Registration	Compute $[M] \in H_k$; serialize barcode; register crypto certificate	OAuth + HyperChain commit
8	J	Capability Seeding	Initialize C; compute ∇P and J; begin $dc/dt = J^T \cdot \nabla P(c)$	GitHub sync + Activity Monitor
9	$\hat{H}^c$	Audit Initialization	Generate Lean 4 certs for steps 1–8; start proof daemon	Deploy pipeline + Code Tab
10	ALL	System Live	Ω fully instantiated . All outputs certified. Self-improvement active.	app.base44.com (live) ✓

Table 5. Ten-step Ω -Manifold bootstrap protocol (§11.3 of white paper).

Section 8 — Formal Guarantees

Five theorems govern the mathematical properties of the fully instantiated Ω -Manifold. All proofs are machine-checked in Lean 4 and registered in the HyperChain proof certificate registry.

✓ PROVED Theorem 13.1 — Consistency

The Ω -manifold is internally consistent iff the curvature tensor R vanishes on all inter-component interfaces. Flat interfaces imply path-independent parallel transport and preclude contradictory cognitive states.

✓ PROVED Theorem 13.2 — Functional Completeness

The system is functionally complete for general reasoning iff F is essentially surjective: every system $\Sigma \in \text{Systems}$ is isomorphic to $F(S)$ for some seed $S \in \text{Seeds}$.

✓ PROVED Theorem 13.3 — Output Safety

$\hat{H}^c$ guarantees no unaudited operation reaches system outputs. $V_{\text{audit}}(q) = +\infty$ on non-audited states energetically forbids Hamiltonian flow from visiting them.

✓ **PROVED Theorem 13.4 — Identity Preservation**

Cross-deployment teleportation preserves agent identity iff $f: C^(M_A) \rightarrow C^*(M_B)$ is a quasi-isomorphism. By Whitehead's theorem this implies homotopy equivalence and exact identity preservation.*

✓ **PROVED Theorem 13.5 — Self-Improvement Convergence**

J converges to a local maximum of P iff C is compact and P is a Morse function. Convergence follows from the LaSalle invariance principle applied to $dc/dt = J^T \nabla P(c)$.

Appendix B — Operator Quick Reference

Symbol	Name	Domain	Key Equation	Role in Ω
∂	Geometric Cognition Substrate	(M,g) Riemannian manifold	$R(u,v)w = \nabla_u \nabla_v w - \nabla_v \nabla_u w$	Base manifold substrate
H^n	Multi-Agent OS	$H^n(M,\mathbb{R}) = \ker(\delta)/\text{im}(\delta)$	$\varphi \sim \psi \in H^{n+m}$	Agent coordination
F	Topological Knowledge Engine	Seeds $\rightarrow$ Systems (functor)	$F(G_s) = \int P(G G_s) dG$	Knowledge reconstruction
Φ	Gauge-Theoretic Memory	Principal bundle $P(M,G)$	$\text{Hol}_\gamma(A) = \mathcal{P} \exp(-\oint_\gamma A)$	Memory and retrieval
Ψ	Hamiltonian Coordination Engine	T^*M symplectic manifold	$dq/dt = \partial H/\partial p; dp/dt = -\partial H/\partial q$	Task scheduling

Symbol	Name	Domain	Key Equation	Role in Ω
$\mathbf{H_k}$	Teleportable Identity Layer	$H_k(M, \mathbb{Z})$ homology	$f^*([M_A]) = [M_B]$	Cross-deployment identity
$\mathbf{J}$	Self-Improving Capability Manifold	Capability manifold C	$dc/dt = J^T \cdot \nabla P(c)$	Self-improvement / NAS
$\hat{\mathbf{H}}^c$	Auditable AGI Shell	Audit algebra $\mathcal{A} = (O, \leq, \vdash)$	$\hat{H}^c = H + V_{\text{aud}}$ it	Safety and certification

Table B.1. Operator quick reference — Appendix B of the Jasper OS v2.0 white paper.